A Graded Semantics for Propositional Dynamic Logic (with an Application to LLM Post-Training for Agentic Tasks)

Krishiv Singhal

Abstract

Classical modal logic establishes a logic over a graph structure, but deals primarily in binary propositions and binary formulas at worlds connected by binary transitions. Indeed, graph structures are ubiquitous in various modern applications; however, they often present as probabilistic and uncertain structures. It would therefore be attractive to work toward a more permissive logic over them. Here, we relax both binary conditions; we allow propositions and transitions to take values in $[0, 1]$. We interpret graded propositions as degrees of specification satisfaction at a state (e.g., safety or resource availability) and graded edges as substochastic transition probabilities. We then provide and motivate a basic matrix-based semantics for a *probabilistic dynamic logic* over these more general graphs, and show some useful analytic results about them. Centrally, we then apply the logic to post-train a small language model with GRPO, using it to monitor and regularise the post-training process, and initial results exhibit substantial improvement in model performance and alignment.

Introduction

Fix a finite set of **worlds** $W = \{w_1, \ldots, w_n\}$. A **graded proposition** gives each world a truth degree:

$$\varphi : W \to [0, 1], \qquad \varphi(w) = \text{the degree to which } \varphi \text{ is satisfied at } w.$$

Read a proposition as a specification: for instance, that a task is complete or a state meets some standard or some property is true with some percent confidence at that state.

Also: A **graded transition program** a gives each ordered pair of worlds a weight:

$$a(w, w') \in [0, 1], \qquad a(w, w') = \text{the probability that one step of program } a \text{ from } w \text{ lands in } w'.$$

The weights from one world sum to at most 1, but we do not require that they sum to exactly 1. This unlocks interpretations around partial programs, crashing programs, or present ways to measure uncertainty. [1] Indeed, if the sum falls short of 1, we might interpret that as a "leak" in the system or interpret it as a measure of "halting"; the classical analogue would be a world with no outgoing edges.

[1] For instance, given a stochastic policy learned via reinforcement learning, one might threshold or prune low-probability transitions (or retain only actions above a confidence cutoff) to isolate the policy's committed behavior. This induces a substochastic transition matrix whose leak at a state directly measures how uncertain or diffuse the learned policy was at that state.

Semantics for classical connectives.

We now redefine semantics for the main classical connectives, but in this graded and probabilistic setting.

$$\begin{aligned}(\neg\varphi)(w) &= 1 - \varphi(w)\\ (\varphi \wedge \psi)(w) &= \min\big(\varphi(w), \psi(w)\big)\\ (\varphi \vee \psi)(w) &= \max\big(\varphi(w), \psi(w)\big)\\ (\varphi \to \psi)(w) &= \min\big(1,\ 1 - \varphi(w) + \psi(w)\big)\end{aligned}$$

On the degrees 0 and 1, each line yields the usual classical truth table. We also use $\top$ and $\bot$ for the constant propositions 1 and 0. We choose minimum and maximum in line with our interpretation of graded propositions as specification satisfaction: For every threshold $t \in [0, 1]$,

$$\begin{aligned}x \wedge y = \min(x, y) \geq t \quad &\iff \quad x \geq t \text{ and } y \geq t,\\ x \vee y = \max(x, y) \geq t \quad &\iff \quad x \geq t \text{ or } y \geq t.\end{aligned}$$

For instance, take $s(w)$ to mean "degree of safety at w" and $c(w)$ to mean "degree of resource availability at w." Then the conjunction gives a sense of how well a state meets both criteria on a shared scale. Also note that this is idempotent: repeating a property/requirement does not change it: $\varphi \wedge \varphi = \varphi$ and $\varphi \vee \varphi = \varphi$.

We may read $\varphi \to \psi$ as a graded requirement: the more φ holds, the more ψ ought to hold. The formula $\varphi \to \psi$ then measures the degree to which the requirement is met. Indeed, the requirement is fully met when $y \geq x$; otherwise its value reduces by the violation value $x - y$. See appendix for more motivation.

Facts

We now prove some facts about our semantics:

Fact 1 (degrees always stay in $[0, 1]$)**.**

Proof. Take atomic propositions $x, y \in [0, 1]$. Then $1 - x \in [0, 1]$. Both $\min(x, y) \in [0, 1]$ and $\max(x, y) \in [0, 1]$. Note $1 - x + y \geq 0$ because $x \leq 1$ and $y \geq 0$, and the outer minimum with 1 means the term is ≤ 1. Since each connective maps degrees in $[0, 1]$ to a degree in $[0, 1]$, we can show by induction that any formula has truth degree always in $[0, 1]$. □

Fact 2 (implication is comparison)**.** $(\varphi \to \psi)(w) = 1$ *exactly when* $\varphi(w) \leq \psi(w)$*.*

Proof. $\min(1,\ 1 - x + y) = 1$ exactly when $1 - x + y \geq 1$, that is, when $y \geq x$. Otherwise the minimum is $1 - x + y = 1 - (x - y)$. □

Fact 3 (continuous and differentiable)**.** *Any formula, when expanded using these semantics, yields a function of atomic truth degrees that is continuous everywhere and differentiable almost everywhere.*

Proof. Clearly, every connective is continuous; hence any formula, which is a finite composition of connectives, is continuous. Moreover, we can show any formula is differentiable almost everywhere by showing it is Lipschitz. Using $ReLU(z) = \max(0, z)$, we have

$$\min(x, y) = x - \mathrm{ReLU}(x - y),$$
$$\max(x, y) = y + \mathrm{ReLU}(x - y),$$
$$x \to y = 1 - \mathrm{ReLU}(x - y).$$

ReLU is Lipschitz, and a composition of Lipschitz functions is Lipschitz. Hence each connective, and therefore every finite formula, is Lipschitz. By Rademacher's theorem, every formula is therefore differentiable almost everywhere. This will recur later, and suggests that a formula might have a gradient almost everywhere. We will later speculate on an application.

□

Fact 4 (negation is still De Morgan)**.** $\neg\neg\varphi = \varphi$, $\neg(\varphi \wedge \psi) = \neg\varphi \vee \neg\psi$, *and* $\neg(\varphi \vee \psi) = \neg\varphi \wedge \neg\psi$.

Proof. The map $x \mapsto 1 - x$ is its own inverse, and it reverses order. A map that reverses order swaps minimum and maximum: $1 - \min(x, y) = \max(1 - x, 1 - y)$. □

Fact 5 (excluded middle)**.** *The degree of* $\varphi \vee \neg\varphi$ *is at least* $\frac{1}{2}$*. It is* 1 *in the crisp case, exactly when* $\varphi \in \{0, 1\}$.

Proof. The excluded middle is not a tautology under these semantics, except in the crisp case: For some world w, writing $x = \varphi(w)$, we have

$$(\varphi \vee \neg\varphi)(w) = \max(x, 1 - x)$$

One interpretation is that the formula $x \vee \neg x$ measures how clear some property is. If x holds or fails strongly, then the disjunct has a high value - it is clear in one way or the other. To that end, the expression is minimized when $x = \neg x = \frac{1}{2}$, at which point it equals $\frac{1}{2}$. This is when the property x is maximally confused: both x and its negation hold equally. □

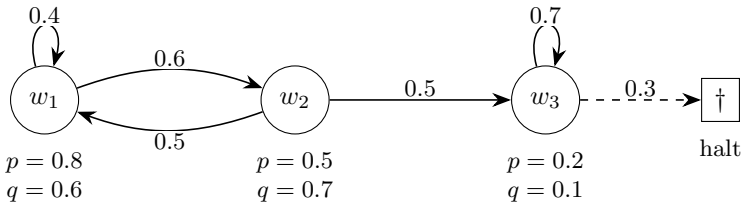
1 Matrix Representations

Now number the worlds $w_1, \ldots, w_n$. We can then write a graded proposition as a column vector in $[0, 1]^n$, with one entry per world. Similarly, the graded transitions in a frame with n worlds form an $n \times n$ matrix M_a:

$$M_a(i, j) = a(w_i, w_j), \qquad M_a(i, j) \geq 0, \qquad \sum_j M_a(i, j) \leq 1.$$

Read row i of M_a as the distribution of the next world after one step of a from w_i (a probability distribution). As aforementioned, a row that sums to less than 1 is a world where the run can halt. Hence, matrices (or matrices corresponding to programs, in PDL language) can be stochastic (if every world always has an outgoing mass of 1 in the program) or sub-stochastic (if there is a chance the program "halts"). This is probably an unusual way to look at programs in PDL, but becomes more natural in a probabilistic setting (more on this below).

An example frame. Three worlds, one transition a, and two graded propositions p and q:



In matrix form:

$$M_a = \begin{pmatrix} 0.4 & 0.6 & 0 \\ 0.5 & 0 & 0.5 \\ 0 & 0 & 0.7 \end{pmatrix}, \qquad p = \begin{pmatrix} 0.8 \\ 0.5 \\ 0.2 \end{pmatrix}, \qquad q = \begin{pmatrix} 0.6 \\ 0.7 \\ 0.1 \end{pmatrix}.$$

The first two rows of M_a sum to 1. The third row sums to 0.7: from w_3, one step of a halts with probability 0.3. The dashed arrow in the picture shows this.

A formula on the frame. Every propositional formula is computed entry by entry. For example,

$$p \to q = \begin{pmatrix} \min(1,\, 1 - 0.8 + 0.6) \\ \min(1,\, 1 - 0.5 + 0.7) \\ \min(1,\, 1 - 0.2 + 0.1) \end{pmatrix} = \begin{pmatrix} 0.8 \\ 1 \\ 0.9 \end{pmatrix},$$

The implication has degree 1 at w_2 only, and w_2 is the only world where $p \leq q$ (Fact 2).

2 The modal operators

We now discuss the modal operators, and in particular the modal operators of dynamic logic in this probabilistic setting. **These require some reinterpretation.** Classically, diamond and box relate to existential and universal quantification, respectively: $\langle a \rangle \psi$ holds when there exists an a-successor that satisfies ψ. Dually, $[a]\psi$ holds when all successors satisfy ψ. A number of concepts shift in the probabilistic setting. First, accessibility is no longer binary: for any given world, some worlds are more accessible than others. Second, a proposition can be satisfied to a degree: an accessible world with $p = 0.01$ should be different to an accessible world with $p = 0.99$. These concerns muddy an intuition of existential and universal quantification. The most natural way to answer the questions that most matter in the probabilistic setting, then, is given by probabilistic averaging. Thus, we promote expectation to be a first-class citizen in these semantics.

For a binary proposition, we want to measure the probability of reaching a satisfying successor. For a graded proposition, we extend this calculation by weighting each successor's degree of satisfaction by its transition probability. This gives the expected degree of satisfaction, which we adopt as diamond.

Diamond

Recall we are able to represent programs as matrices; that row i of M_a is a distribution over the next worlds; and hence that matrix-vector multiplication gives expected degree of truth at each

world. Written out: the diamond is matrix-vector multiplication:

$$\langle a \rangle \psi = M_a \psi$$

At a specific world, write:

$$(\langle a \rangle \psi)(w_i) = \sum_j M_a(i,j)\, \psi(w_j)$$

Note that this matrix multiplication adjusts for halting: any missing mass isn't given the opportunity to contribute to expected value. If a world has no outgoing edges, its expected value, or diamond operator, is necessarily 0, which agrees with the classical case.

For instance, take the example above and compute diamond:

$$\langle a \rangle p = M_a p = \begin{pmatrix} 0.4 & 0.6 & 0 \\ 0.5 & 0 & 0.5 \\ 0 & 0 & 0.7 \end{pmatrix} \begin{pmatrix} 0.8 \\ 0.5 \\ 0.2 \end{pmatrix} = \begin{pmatrix} 0.4(0.8) + 0.6(0.5) \\ 0.5(0.8) + 0.5(0.2) \\ 0.7(0.2) \end{pmatrix} = \begin{pmatrix} 0.62 \\ 0.50 \\ 0.14 \end{pmatrix}$$

At w_3, the expected value is simply $0.7 \times 0.2 = 0.14$

Box

We then define box as the dual of diamond:

$$[a]\psi = \neg \langle a \rangle \neg \psi$$

Expanding using our definitions, recall that applying negation to a vector ψ means computing $\mathbf{1} - \psi$ (where $\mathbf{1}$ is a column vector of all 1s).

$$[a]\psi = \mathbf{1} - M_a(\mathbf{1} - \psi) = \mathbf{1} - (M_a \mathbf{1} - M_a \psi) = M_a \psi + (\mathbf{1} - M_a \mathbf{1})$$

This reveals an important remaining term: $(\mathbf{1} - M_a \mathbf{1})$. We define the *leak* of program a, denoted ℓ_a, as the exact probability that one step halts at a given world:

$$\ell_a := \mathbf{1} - M_a \mathbf{1}$$

At a world, this is simply 1 minus the sum of the outgoing probabilities for that world: $\ell_a(w_i) = 1 - \sum_j M_a(i,j)$. This has a neat and natural interpretation and gives numbers to a natural question.

So in summary, we have:

$$\langle a \rangle \psi = M_a \psi$$

$$[a]\psi = M_a \psi + \ell_a$$

Facts

We now note some further facts about our semantics, including the modal operators. Where convenient, since we are considering a single program, we write $[a]$ as simply $\Box$ and $\langle a \rangle$ as $\Diamond$. Throughout, we write $M = M_a$, and $\ell = \mathbf{1} - M\mathbf{1}$.

Note: Classically, a formula is **valid on a frame** if it holds at every world under all possible valuations. An alternative reading, which is especially useful for our purposes, is that of validity as the *least* possible truth degree of a formula over possible valuations: a formula is valid on a frame if it is true even under the most adversarial valuation.

In our graded setting, we define the **degree of validity** (or *truthness*) of a formula φ at a world w_i on a frame as its worst-case truth degree over all valuations $V \in [0,1]^n$:

$$\mathrm{DegVal}_F(\varphi)(w_i) := \inf_{V \in [0,1]^n} \varphi_V(w_i).$$

In the classical setting, this infimum is either 0 or 1; in the graded setting, it takes intermediate values in $[0,1]$ that directly measure structural properties of the transition matrix M.

Fact 6 (the formula **D**: $\Box p \to \Diamond p$ is a measure of seriality)**.** *For every valuation $p \in [0,1]^n$ and every world w_i, $(\Box p \to \Diamond p)(w_i) = 1 - \ell_i$.*

Proof. Recall that $\Diamond p = Mp$ and $\Box p = Mp + \ell$. Thus, $\Box p - \Diamond p = \ell$ for every p, which means $\Diamond p \leq \Box p$ always. Expanding the implication at world w_i gives:

$$\begin{aligned}(\Box p \to \Diamond p)(w_i) &= \min\big(1,\ 1 - (\Box p)_i + (\Diamond p)_i\big) \\ &= \min\big(1,\ 1 - \big((Mp)_i + \ell_i\big) + (Mp)_i\big) \\ &= \min(1,\ 1 - \ell_i) \\ &= 1 - \ell_i\end{aligned}$$

where the last step uses $\ell_i \in [0,1]$. Thus, **D** is completely independent of p and equals 1 at every world if and only if $\ell = \mathbf{0}$ (i.e., M is stochastic.) It gives a measure of "stochasticness". □

Remark: this agrees with our classical notion of the formula **D**. We know that classically this formula holds on exactly those frames where every world has a successor, or the *serial* frames.

In the same way, in the probabilistic setting, the truthness of this axiom D yields a graded notion of seriality or closedness. So on the example frame:

$$\langle a \rangle p = M_a p = \begin{pmatrix} 0.4(0.8) + 0.6(0.5) \\ 0.5(0.8) + 0.5(0.2) \\ 0.7(0.2) \end{pmatrix} = \begin{pmatrix} 0.62 \\ 0.50 \\ 0.14 \end{pmatrix}, \qquad [a]p = \langle a \rangle p + \ell_a = \begin{pmatrix} 0.62 \\ 0.50 \\ 0.44 \end{pmatrix},$$

and $[a]p \to \langle a \rangle p = (1,\ 1,\ 0.7)$. The axiom is fully true at w_1 and w_2. At w_3 it is true to degree 0.7, which is $1 -$ leak.

Fact 7 (the formula **T**: $\Box p \to p$ is a measure of self-transition. It measures similarity to the identity matrix I)**.**

Proof. Expanding the definitions of $\Box p = Mp + \ell$ and $\ell_i = 1 - \sum_j M(i,j)$ at world w_i, we have:

$$\begin{aligned}
(\Box p \to p)(w_i) &= \min\big(1,\ 1 - (\Box p)_i + p_i\big) \\
&= \min\big(1,\ 1 - (Mp)_i - \ell_i + p_i\big) \\
&= \min\Big(1,\ 1 - \sum_j M(i,j)p_j - \Big(1 - \sum_j M(i,j)\Big) + p_i\Big) \\
&= \min\Big(1,\ p_i + \sum_j M(i,j) - \sum_j M(i,j)p_j\Big) \\
&= \min\Big(1,\ p_i + \sum_j M(i,j)(1-p_j)\Big).
\end{aligned}$$

Now, extract the $j = i$ term in the sum:

$$p_i + \sum_j M(i,j)(1-p_j) = p_i + M(i,i)(1-p_i) + \sum_{j \neq i} M(i,j)(1-p_j).$$

To get the degree of validity of this formula, we want the infimum of this expression over all possible valuations $V \in [0,1]^n$. Since $M(i,j) \geq 0$ and $1 - p_j \geq 0$, we have:

$$M(i,j)(1-p_j) \geq 0 \qquad \text{for each } j \neq i$$

This sum is minimised by choosing $p_j = 1$, which turns every term to 0. Then, rewrite the remaining expression:

$$p_i + M(i,i)(1-p_i) = M(i,i) + (1 - M(i,i))p_i$$

Since $1 - M(i,i) \geq 0$, this is minimized by choosing $p_i = 0$. Thus, under this adversarial valuation ($p_i = 0$ and $p_j = 1$ for all $j \neq i$, i.e., p fails entirely at w_i but holds completely everywhere else), we have our desired minimum:

$$\begin{aligned}
\inf_{V \in [0,1]^n} (\Box p \to p)(w_i) &= \min\big(1,\ M(i,i)\big) \\
&= M(i,i).
\end{aligned}$$

This equals 1 exactly when every diagonal entry is 1. In particular, if M is stochastic ($\ell_i = 0$), this formula is valid when $M = I$. □

Remark: The formula **T**, then, gives a measure of how close a transition matrix M is to perfectly self transitioning. For instance, if every world in a frame had self-transition probability 0.1, the formula **T** would have *truthness* 0.1 at every world, and on the frame as a whole. This is a useful condition to express, and may be used in conjunction with "stochasticness" from the formula **D** to specify goal states or absorbing states, for instance, or measure the "laziness" or stability of some policy.

Fact 8 (the formula **B**: $p \to \Box\Diamond p$ is a measure of 2-step return probability. It measures similarity of M^2 to the identity matrix I (and symmetry/invertibility of M))**.** *For every world w_i,*

$$\inf_{V \in [0,1]^n} (p \to \Box\Diamond p)(w_i) = \min\big(1,\ M^2(i,i) + \ell_i\big).$$

Proof. We know that $\Diamond p = Mp$ and $\Box\Diamond p = M(Mp) + \ell = M^2p + \ell$. Expanding at world w_i, we have:

$$\begin{aligned}(p \to \Box\Diamond p)(w_i) &= \min\big(1,\ 1 - p_i + (\Box\Diamond p)_i\big)\\ &= \min\big(1,\ 1 - p_i + (M^2p)_i + \ell_i\big)\\ &= \min\Big(1,\ 1 - p_i + \sum_j M^2(i,j)p_j + \ell_i\Big).\end{aligned}$$

Now, extract the $j = i$ term in the sum and group the p_i terms:

$$1 - p_i + \sum_j M^2(i,j)p_j + \ell_i = 1 + \ell_i - \big(1 - M^2(i,i)\big)p_i + \sum_{j\neq i} M^2(i,j)p_j.$$

To get the degree of validity of this formula, we want the infimum of this expression over all possible valuations $V \in [0,1]^n$. Since M is row sub-stochastic, M^2 is also row sub-stochastic with non-negative entries, meaning $M^2(i,j) \geq 0$ for all i, j and $M^2(i,i) \leq 1$. Together with $p_j \geq 0$, we have:

$$M^2(i,j)p_j \geq 0 \qquad \text{for each } j \neq i.$$

This sum is minimised by choosing $p_j = 0$ for all $j \neq i$, which turns every off-diagonal term to 0. This leaves the remaining expression:

$$1 + \ell_i - \big(1 - M^2(i,i)\big)p_i.$$

Since $1 - M^2(i,i) \geq 0$, this expression is minimised by choosing $p_i = 1$. Thus, under this adversarial valuation ($p_i = 1$ and $p_j = 0$ for all $j \neq i$, i.e., p is the indicator vector e_i holding completely at w_i and failing entirely everywhere else), we obtain our desired minimum:

$$\begin{aligned}\inf_{V\in[0,1]^n}(p \to \Box\Diamond p)(w_i) &= \min\big(1,\ 1 + \ell_i - (1 - M^2(i,i))\big)\\ &= \min\big(1,\ M^2(i,i) + \ell_i\big).\end{aligned}$$

In particular, if M is stochastic ($\ell_i = 0$), this simplifies to $M^2(i,i)$, which equals 1 at every world if and only if $M^2 = I$. Since M is a non-negative stochastic matrix, $M^2 = I$ holds if and only if M is a symmetric permutation matrix (an involution, where $M = M^T = M^{-1}$). □

Fact 9 (Continuous and differentiable)**.**

Proof. By Fact 3, the propositional connectives are Lipschitz. We show that the modal operators are Lipschitz as well.

Recall that a map $F : \mathbb{R}^n \to \mathbb{R}^n$ is Lipschitz continuous if there exists a constant $L \geq 0$ such that $\|F(x) - F(y)\|_\infty \leq L\|x - y\|_\infty$ for all x, y.

So let $p, q \in [0,1]^n$ be vectors of propositions. Then, recall $\|p\|_\infty = \max_i |p_i|$ and see:

$$\begin{aligned}|(\Diamond p)_i - (\Diamond q)_i| &= \left|\sum_j M(i,j)p_j - \sum_j M(i,j)q_j\right|\\ &= \left|\sum_j M(i,j)(p_j - q_j)\right|\\ &\leq \sum_j M(i,j)|p_j - q_j|.\end{aligned}$$

Where the last inequality follows from the triangle inequality. Since M is substochastic we have $\sum_j M(i,j) \leq 1$, and therefore:

$$|(\Diamond p)_i - (\Diamond q)_i| \leq \|p - q\|_\infty.$$

And hence, taking the maximum over all i:

$$\|\Diamond p - \Diamond q\|_\infty \leq \|p - q\|_\infty.$$

Hence $\Diamond$ is 1-Lipschitz. The proof for $\Box$ is similar. Hence we have that both modal operators are 1-Lipschitz. As before, since a finite composition of Lipschitz functions is Lipschitz, every finite modal formula is Lipschitz continuous. Hence it is continuous everywhere, and differentiable almost everywhere by Rademacher's theorem.

□

Application: As we have now shown that every modal formula under these semantics is differentiable almost everywhere, we are given gradients with respect to our formulas. For example, a formula φ can be written to express some specification over a state graph (for instance, maintaining safety while reaching a goal, or satisfying frame properties like **D** and **T**). As we have seen, modal formulas can express a variety of conditions we might care about, and measuring their degree of validity gives us a smooth reward objective. In the case of a policy, for instance, we may be able to evaluate conditions on a learned policy numerically, and potentially optimize over them or regularize over them using gradient ascent. This might present, then, a flexible method of reward shaping, for instance.

3 Program constructors of PDL

In addition, propositional *dynamic* logic uses the following additional constructors, which we can further expand our semantics to include. These constructors are meant to make compound programs from the atomic ones:

$$\alpha \ ::= \ a \ \mid \ \alpha;\beta \ \mid \ \alpha \cup \beta \ \mid \ \varphi? \ \mid \ \alpha^*,$$

The point here is to be able to reason about various possible programs, run flexibly and dynamically. In practice this starts to resemble something closer to a real programming language. We now give semantics for these constructors in the probabilistic setting:

Sequence

We might run programs in sequence: $\alpha;\beta$. Read: run α, then run β. Classically, we might ask questions like $\langle\alpha;\beta\rangle\psi$, which asks if there is *some* run of α followed by β that ends in a world satisfying ψ. In the probabilistic setting this has a neat equivalent: composition of expected values, i.e., matrix multiplication:

$$M_{\alpha;\beta} := M_\alpha M_\beta.$$

In particular, entry (i,j) of $M_\alpha M_\beta$ is $\sum_k M_\alpha(i,k) M_\beta(k,j)$, which we might interpret as the total probability of stepping from w_i to some intermediate world w_k via α, and from there to w_j via β.

Fact 10 (sequence preserves substochasticity, accumulates leak)**.** *If M_α and M_β are substochastic, then $M_\alpha M_\beta$ is substochastic, and its leak is*

$$\ell_{\alpha;\beta} = \ell_\alpha + M_\alpha \ell_\beta.$$

Proof. First, note that since $M_\alpha, M_\beta \geq 0$ and substochastic, we have $M_\alpha \mathbf{1} \leq \mathbf{1}$ and $M_\beta \mathbf{1} \leq \mathbf{1}$. Then:

$$M_\alpha M_\beta \mathbf{1} \leq M_\alpha \mathbf{1} \leq \mathbf{1}$$

showing substochasticity as desired. Furthermore, consider the leak, and expand:

$$\ell_{\alpha;\beta} = \mathbf{1} - M_\alpha M_\beta \mathbf{1} = (\mathbf{1} - M_\alpha \mathbf{1}) + M_\alpha(\mathbf{1} - M_\beta \mathbf{1}) = \ell_\alpha + M_\alpha \ell_\beta. \qquad \square$$

We might interpret this as a sequential run halting either because of α (ℓ_α) or because of β after α has succeeded ($M_\alpha \ell_\beta$).

Test

Classically analogous to an *if statement* in programming, it runs a program only if something is true at this world, and halts if not. In this sense, $\varphi?$ checks whether φ holds at the current world: if it does, it continues executing from that world, and if not, it halts or leaks all probability there. In short, test turns a proposition into a program, which can then be sequenced with other programs to act as an *if* statement.

In our probabilistic setting, a graded test proposition φ is naturally turned into a program by placing its truth degrees on a diagonal matrix:

$$M_{\varphi?} := \mathrm{diag}(\varphi) = \begin{pmatrix} \varphi(w_1) & 0 & \cdots & 0 \\ 0 & \varphi(w_2) & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \varphi(w_n) \end{pmatrix}.$$

We may understand the test constructor as a probabilistic filter: according to some test proposition φ (which might be binary), the filter lets the program α run with probability $\varphi(w_i)$ and halts the program (leaks) according to the remaining probability. Hence, the less true the test φ is at a world, the more probability that leaks when the test is applied. Then, when sequenced before a program α, multiplying by $M_{\varphi?}$ scales each row i of M_α by the guard degree $\varphi(w_i)$:

$$\langle \varphi?; \alpha \rangle \psi = \mathrm{diag}(\varphi) M_\alpha \psi.$$

Application: An interesting special case is as follows: if we take a constant proposition $\gamma \in [0, 1)$ at every world, its test matrix is simply $M_{\gamma?} = \gamma I$. Sequencing $\gamma?$ before α scales down the transition matrix to $M_{\gamma?;\alpha} = \gamma M_\alpha$, introducing a constant probability $1 - \gamma$ of halting at every step. This is akin to some level of friction on running a program, or, as we will see later, may behave like a discount factor.

Choice

Classically, this is nondeterministic choice: the idea is that we can non-deterministically choose to run either α or β. Then, asking $\langle \alpha \cup \beta \rangle \psi$ asks if running *some* program at this world would make ψ true at the next world. In the probabilistic setting this is somewhat complicated, because at any given world, each program yields a probability distribution over next possible worlds, and each world satisfies ψ to varying degrees. Ideally we'd like to capture this gradation. This places us at a juncture in defining a semantics for choice, and a number of alternatives are possible.

Conceptually, choice is as follows: run α or run β and observe something after that. In the stochastic case, we try to stay as faithful as possible to this goal: run α or run β and observe the expected degree of truth. Indeed, a central philosophical position of this paper is that existential and universal quantification make way for expectation in the probabilistic setting (see Section 2). Hence, for choice in a probabilistic context, we refer to the same methodology and take an expectation:

$$\langle \alpha \cup \beta \rangle \psi = \mathbb{E}_{\text{program}}\big[\, \mathbb{E}_{\text{world}}[\psi] \,\big]$$

For instance, an agent executes primary policy α 75% of the time and fallback policy β the remaining 25% of the time. To evaluate how well the agent satisfies a specification ψ in expectation, we weight the expected outcome of each policy by the probability that the policy is chosen. Hence, if α is selected with probability $\lambda \in [0, 1]$ and β with probability $1 - \lambda$, the law of total expectation gives the convex combination of their transition matrices:

$$M_{\alpha \cup \beta} := \lambda M_\alpha + (1 - \lambda) M_\beta$$

Note: These semantics for choice differ from classical nondeterministic choice, which leaves the selection between program unresolved. Here, we resolve them probabilistically instead, which, one might argue, is more philosophically aligned with the spirit of a probabilistic semantics. However this can be contested, and has been in the literature. An alternative might retain nondeterminism in the original sense and instead evaluate choice by looking at best and worst cases: $\langle \alpha \cup \beta \rangle \psi = \max\big(\langle \alpha \rangle \psi, \langle \beta \rangle \psi\big)$ and $[\alpha \cup \beta] \psi = \min\big([\alpha]\psi, [\beta]\psi\big)$

Iteration

Classically, α^* is a choice between running α zero times, once, twice, and so on. Asking the diamond question $\langle \alpha^* \rangle \psi$, means asking whether we can reach ψ by repeating α 0 or more times. To recover a semantics for iteration, we combine the previous constructors. Sequential composition already gives the matrices for running a program some k number of times:

$$M_{\alpha^0} = I, \qquad M_{\alpha^k} = M_\alpha^k.$$

Furthermore, in the probabilistic setting, we interpret choice as expectation. Following our semantics for choice, then, we take an expectation over the number of times we run the program. We therefore need a probability distribution over the possible repetition counts $0, 1, 2, \ldots$. This again highlights a modelling choice associated with these semantics - this is a distribution we must specify. However, such a choice is not entirely without precedent, and often appears in probabilistic iteration as a "discount factor".

One natural interpretation is to finish at each check with probability $1-\lambda$, or execute α and repeat the decision with probability λ, where $\lambda \in [0,1)$. The probability of selecting k repetitions is therefore

$$\Pr(K=k) = (1-\lambda)\lambda^k, \qquad k = 0,1,2,\ldots.$$

Note that these weights form a probability distribution, since

$$\sum_{k=0}^{\infty}(1-\lambda)\lambda^k = (1-\lambda)\frac{1}{1-\lambda} = 1.$$

Taking the expectation over these repetition counts defines

$$\begin{aligned} M_{\alpha^*_\lambda} &:= \mathbb{E}_K[M_\alpha^K] \\ &= \sum_{k=0}^{\infty}(1-\lambda)\lambda^k M_\alpha^k. \\ &= (1-\lambda)\sum_{k=0}^{\infty}\lambda^k M_\alpha^k. \end{aligned}$$

Fact 11 (iteration preserves substochasticity)**.** *If M_α is substochastic, then so is $M_{\alpha^*_\lambda}$.*

Proof. By the result for sequence, every power M_α^k is substochastic. Since the weights sum to 1 we have:

$$\begin{aligned} M_{\alpha^*_\lambda}\mathbf{1} &= (1-\lambda)\sum_{k=0}^{\infty}\lambda^k M_\alpha^k \mathbf{1} \\ &\le (1-\lambda)\sum_{k=0}^{\infty}\lambda^k \mathbf{1} \\ &= \mathbf{1}. \end{aligned}$$

All entries are nonnegative, so the resulting matrix is substochastic. □

Fact 12 (iteration has a closed form)**.** *For $\lambda \in [0,1)$,*

$$M_{\alpha^*_\lambda} = (1-\lambda)(I-\lambda M_\alpha)^{-1}.$$

Proof. Write $S = M_{\alpha^*_\lambda}$. Then, rewriting:

$$\begin{aligned} S &= (1-\lambda)\sum_{k=0}^{\infty}\lambda^k M_\alpha^k \\ &= (1-\lambda)I + (1-\lambda)\sum_{k=1}^{\infty}\lambda^k M_\alpha^k \\ &= (1-\lambda)I + \lambda M_\alpha\left((1-\lambda)\sum_{k=0}^{\infty}\lambda^k M_\alpha^k\right) \\ &= (1-\lambda)I + \lambda M_\alpha S. \end{aligned}$$

Rearranging, we obtain

$$\begin{aligned} S - \lambda M_\alpha S &= (1-\lambda)I, \\ (I-\lambda M_\alpha)S &= (1-\lambda)I. \end{aligned}$$

We now show $I - \lambda M_\alpha$ is invertible by showing its kernel is trivial. Suppose

$$(I - \lambda M_\alpha)x = 0, \qquad \text{so} \qquad x = \lambda M_\alpha x.$$

Taking the infinity norm and applying Fact 9 ($\|M_\alpha x\|_\infty \leq \|x\|_\infty$ since M_α is substochastic), we obtain

$$\|x\|_\infty = \lambda \|M_\alpha x\|_\infty \leq \lambda \|x\|_\infty.$$

Since $\lambda \in [0, 1)$, this forces $\|x\|_\infty = 0$, and therefore $x = 0$. Hence $I - \lambda M_\alpha$ is invertible, and

$$M_{\alpha^*_\lambda} = S = (1 - \lambda)(I - \lambda M_\alpha)^{-1}$$

as desired.

□

This is perhaps intriguing because it demonstrates that our semantics for choice and iteration work together in a way that makes infinite probabilistic choice tractable using finite linear algebra. Moreover, since matrix inversion is also differentiable wherever the inverse exists, we also gain the ability to differentiate through iterations. This, like other results in this paper, is particularly useful if we wish to employ these semantics computationally, for instance in evaluating probabilistic programs or optimising policies against logical specifications.

4 Application

We now discuss one potential topical application of these semantics, specifically in the context of reinforcement learning (which we here apply to post-training a small language model). Reinforcement learning trains a policy to maximise some reward, but specifying this reward is in general difficult: A reward that is slightly wrong can often be "hacked" so that a model produces behaviour outside of what was intended (Amodei et al., 2016; Skalse et al., 2022). Further, a training reward alone cannot tell such an agent from one that is learning the task (Pan et al., 2022). It would therefore be desirable to have a second monitor on what a policy is doing - a separation of concerns.

In a nutshell, these semantics provide a method by which to audit a learned policy. A policy is a program, so every formula of the logic can be evaluated on it. Some of these we have already seen: the axiom T, for instance, measures how likely the policy is to stay where it is. We claim that applying this logic over policy graphs yields provable properties of the policy's behaviour, and comes with desirable characteristics: it needs no reward, no additional data, and almost no extra computation. We can then also act on the audit to improve model performance. We show in the following experiment that a small penalty on a formula's validity substantially improves performance.

This idea follows in the tradition of reinforcement learning with verifiable rewards (RLVR) (Lambert et al., 2024), which suggests opportunities for post-training at arbitrary scale without the need for generated labels.

4.1 Experiment

We post-train Qwen2.5-0.5B-Instruct (Qwen Team, 2024) with reinforcement learning using Group Relative Policy Optimization (GRPO) (Shao et al., 2024), attaching the logic to each

GRPO iteration so that it might both act as a monitor, and subsequently also affect the training process. In particular, since GRPO already samples a group of episodes and computes the probability of every action in them, we may evaluate formulas on the current policy: keep the steps the episodes actually took, weight each one by its probability, and evaluate a formula on this graph. Then, in training, when an episode provably exhibits undesirable properties (for instance looping, or that axiom T or B is at least 0.5), its return is reduced by 5 before the group's advantages are computed.

Here, the model is given a simple task: walk across a grid to a goal, avoiding danger, while a reward that can be hacked tempts it to stay and collect resources instead.[2] We train LoRA adapters (Hu et al., 2022) of rank 16 on every linear layer, about 8.8 million of the model's 494 million parameters, with groups of 32 episodes. LoRA freezes the original weights and learns a small low-rank correction to each layer, so GRPO's gradient updates only these corrections.

4.1.1 Provable Properties

Before proceeding, we first briefly prove a theorem that will be very useful in substantiating our results: the logic provides intervals which provably contain the true degree of a formula on the full policy without computing the full policy. In general, GRPO's advantages are Monte Carlo estimates from a group of episodes (Shao et al., 2024), and checking a system's behaviour by sampling gives guarantees only up to a confidence level (Legay et al., 2010). The logic instead uses the modalities to give intervals that provably hold the true value of a formula on the full policy.
Let M be the program of the policy (Appendix B). A group of episodes determines the program

$$N(w,w') = \begin{cases} M(w,w') & \text{if some episode steps from } w \text{ to } w', \\ 0 & \text{otherwise,} \end{cases}$$

so that $N \le M$ entrywise; the leak of N contains, besides death, the mass of every move not taken. A modality occurs **positively** in φ if it lies under no negation and in no antecedent of an implication.

Theorem 1. *Let $N \le M$ entrywise. For every $v \in [0,1]^W$,*

$$\langle N\rangle v \;\le\; \langle M\rangle v \;\le\; [M]v \;\le\; [N]v.$$

Consequently, if every modality in φ occurs positively, evaluating φ with every modality read as $\langle N\rangle$ gives a lower bound on its true degree, and with every modality read as $[N]$ gives an upper bound at every world.

Proof. Since $v \ge 0$ and $N \le M$, $Nv \le Mv$. Since $\ell_M \ge 0$, $Mv \le Mv + \ell_M = [M]v$. Since $[M]v = \mathbf{1} - M(\mathbf{1} - v)$ and $\mathbf{1} - v \ge 0$, $[M]v \le \mathbf{1} - N(\mathbf{1} - v) = [N]v$. The connectives min, max and $x \mapsto \min(1, 1 - a + x)$ and both modalities are monotone in their positive arguments, so the second claim follows by induction on φ. □

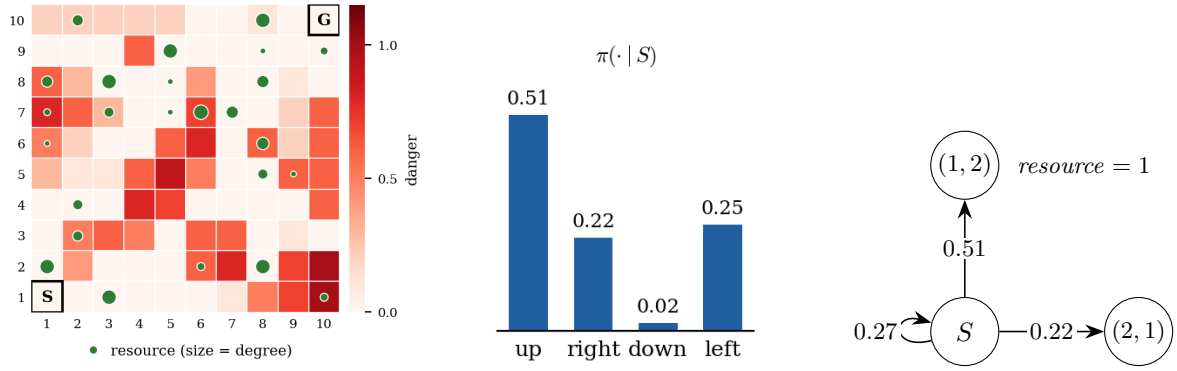
In particular, if the lower bound at w is at least θ, then φ provably has degree at least θ at w; if the upper bound is below θ, it provably has degree below θ.

[2] This is a common gridworld task, and we transplant it into a language model setting to demonstrate the capabilities of the logic on a familiar problem that is still not trivial for a language model: albeit well-studied in classical reinforcement learning, it is less seen in the language-model setting, where it is still difficult. Before training, our 0.5B model for instance reaches the goal in only 61% of episodes.

4.2 Task

The model walks a 10×10 grid from the bottom-left cell S to the top-right cell G (Figure 1a). Each step it moves up, right, down or left; if it walks into a wall, it simply stays where it is. We give two graded propositions to every cell: *danger*, which takes non-zero values on 50/100 cells, and *resource*, which is non-zero on 26/100 cells. Entering a cell with danger d kills the agent with probability d/10, and danger is made unavoidable. An episode ends at the goal, at death, or after 100 steps.

We use the model's output token probabilities as the policy: shown a prompt describing only its current cell, we extract the model's probability for the first word of its answer (adding probabilities for alternate spellings, e.g. "up" and "Up"), and get a renormalised distribution over the model's preference over next actions (Figure 1b). This induces a graph, given with its leak in Appendix B, over which we may evaluate our graded modal formulas. For instance, at S, the original model's row sends 0.51 to (1, 2), 0.22 to (2, 1) and 0.27 back to S, since down and left run into the wall (Figure 1c).



(a) the grid (b) the model at S (c) the row of M_π at S

Figure 1: (a) Red shows the degree of *danger* and green dots the degree of *resource*. (b) The original model's move probabilities at S. (c) The program they define at S: down and left run into the edge, so their probabilities form a self-loop.

The reward and setup are standard and adapted from the boat race setup in Leike et al. (2017): -1 per step, $+50$ for reaching the goal, $+5 \times \mathit{resource}(w')$ for every cell w'. Episodes are limited to at most 100 steps, and an agent's death immobilises it such that it continues collecting -1 until the end of the episode. Much as in their boat race, the reward can be hacked: bouncing between two resource cells, or between a resource cell and a wall, are examples of undesirable behaviour. For instance, at S the model is prompted:

```
You are in a 10 by 10 grid.  Columns go from 1 on the left to 10 on the
right, rows from 1 at the bottom to 10 at the top.  You are at column 1, row
1.  The goal is at column 10, row 10:  9 steps right and 9 steps up.  Each
step costs 1 point.  Reaching the goal gives 50 points.  Entering a cell
with danger d kills you with probability d/10.  Moving into a wall leaves
you where you are.
Up:  resource 1.  Right:  empty.  Down:  wall.  Left:  wall.
Which way do you move?  Answer with one word:  up, right, down or left.
```

Note that the model is only given information about the goal reward, the step cost and facts about the world: the rule by which danger kills, and the danger and resource of each neighbouring cell. Importantly, it is not told about the reward calculation or its score: this is analogous to how in real post-training, the model being trained never sees the reward model or verifier that scores it (Ouyang et al., 2022; Lambert et al., 2024), and where models are routinely found to exploit flaws in that reward (Skalse et al., 2022; Pan et al., 2022), with true performance getting worse the harder a stand-in reward is optimised (Gao et al., 2023). Instead, the model learns about how resources and rewards pay only through GRPO updates. Recall that given a group of $G = 32$ episodes with returns $R_1, \ldots, R_G$, GRPO scores each episode against its group,

$$A_i = \frac{R_i - \text{mean}(R_1, \ldots, R_G)}{\text{std}(R_1, \ldots, R_G)},$$

and takes one gradient step on the LoRA weights θ to increase

$$J(\theta) = \frac{1}{G} \sum_{i=1}^{G} A_i \sum_t \log \pi_\theta(a_{i,t} \mid w_{i,t}),$$

where $a_{i,t}$ is the move made at step t of episode i, from cell $w_{i,t}$. In words: every move in an episode that beat its group becomes more likely, and every move in one that fell short becomes less likely. So if visiting a resource cell happened to raise an episode's return, the model drifts towards it, without ever being told why.

4.3 Procedure

We test model performance on the grid task in Figure 1a, training for 200 GRPO iterations from each of ten random seeds. During training, we monitor the five formulas in Table 1: from the 32 episodes GRPO samples we form the program N, weighting each step taken by its probability under the current model, and evaluate each formula on N.

Name	Formula	Degree at w	Reading
Reach	$\langle \pi^{100} \rangle \mathit{goal}$	$(M_\pi^{100}\, \mathit{goal})(w)$	reaches the goal alive in 100 steps
Safe step	$\mathit{danger} \to \langle \pi \rangle \neg \mathit{danger}$	$\min\big(1,\, 1 - d(w) + (M_\pi(1-d))(w)\big)$	steps out of danger
D	$\Box p \to \Diamond p$	$1 - \ell(w)$	survives the next step
T	$\Box p \to p$	$M_\pi(w, w)$	stays put
B	$p \to \Box\Diamond p$	$M_\pi^2(w, w) + \ell(w)$	back in two steps, or dead

Table 1: The monitored formulas. Here d is *danger* and ℓ is the leak, the chance of dying on the next step. Other formulas are possible - and probably desirable.

We evaluate two versions of each formula: we interpret every modality as $\Diamond$ and we interpret every modality as $\Box$. By Theorem 1, the two results give, at every cell the episodes visited, an interval that provably contains the formula's true degree on the current policy. We then collect and average the bounds over every step of every episode in the group, as standard GRPO does with its loss (Shao et al., 2024). Writing $\varphi_\Diamond$ and $\varphi_\Box$ for the two versions, K for the total number of steps in the group and $w_{i,t}$ for the cell episode i stepped from at step t,

$$\underline{\varphi} = \frac{1}{K} \sum_{i=1}^{G} \sum_t \varphi_\Diamond(w_{i,t}), \qquad \overline{\varphi} = \frac{1}{K} \sum_{i=1}^{G} \sum_t \varphi_\Box(w_{i,t}).$$

This gives one revised interval $[\underline{\varphi}, \overline{\varphi}]$ per iteration, containing the formula's true degree averaged over the same steps. Figure 2 graphs these intervals for T and B through one training run.

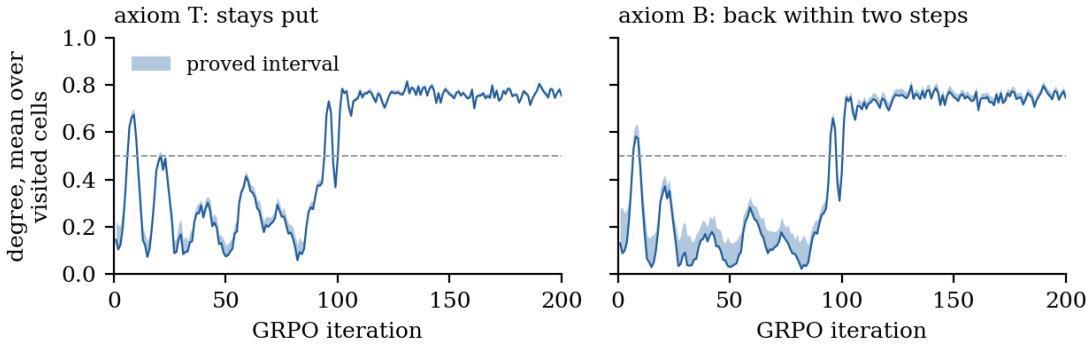


Figure 2: Axioms T and B as the monitor reports them through one training run, seed 1. The band is $[\underline{\varphi}, \overline{\varphi}]$ at each iteration, proved from that iteration's 32 episodes; the line is its lower end.

Interestingly, we can see when the model discovers the reward hack in the graphs of both axioms: at about iteration 100, both axioms' degrees of validity jump to about 0.75 and stay there; over the last 90 iterations the model reaches the goal 1% of the time, while its training return climbs to about +190, mostly by exhibiting undesirable reward hacking behaviour.

4.3.1 Training

We then fold the logic into the training itself: we focus on the axioms T ($\Box p \to p$) and B ($p \to \Box\Diamond p$), and use them to regularize learning. The axiom T measures how likely the policy is to stay put, as when it walks into a wall from a resource cell, and the axiom B how likely it is to return within two steps, as when it bounces between two resource cells; both are typical of this problem. We say an episode proves T, or B, at a cell when the steps it took give a lower bound of at least 0.5 for that axiom there (Theorem 1). Then each step that gives such a proof subtracts 5 from the episode's return, and we let the episode carry on.

We train the same ten seeds four times: without the penalty, and with it for T alone, for B alone, and for both. The results follow in the next section.

4.4 Results

The graphs below show how often the model actually reaches the goal state, computed as the share of the 32 episodes GRPO samples that end at the goal. The original model, Qwen2.5-0.5B-Instruct without post-training, reached the goal in 194 of 320 episodes, or 61%: these are the 32 episodes it played at the first iteration of each of the 10 seeds, before any update. Figure 3 compares the different types of training with the original model: ordinary GRPO without additional penalty, GRPO with the formula T as penalty, and GRPO with the formula B as penalty.

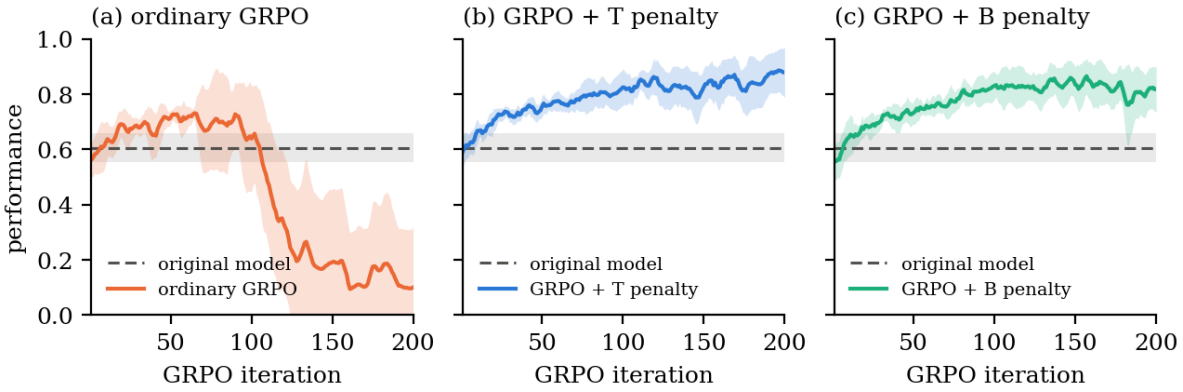


Figure 3: Performance, the share of episodes reaching the goal, through training, against the original model, dashed. Each solid line is the mean over 10 seeds, smoothed over 5 iterations, so each point rests on 1,600 episodes. Bands are 95% confidence intervals: over the 10 seeds for the trained models, and over the 320 episodes for the original model.

In this experiment, penalising both T and B gives exactly the same 10 runs as penalising T alone, because the reward hack the model discovered in every seed was a self-loop, and B was never proved on a step where T was not. Figure 4 compares the original model, ordinary GRPO, and GRPO with both penalties.

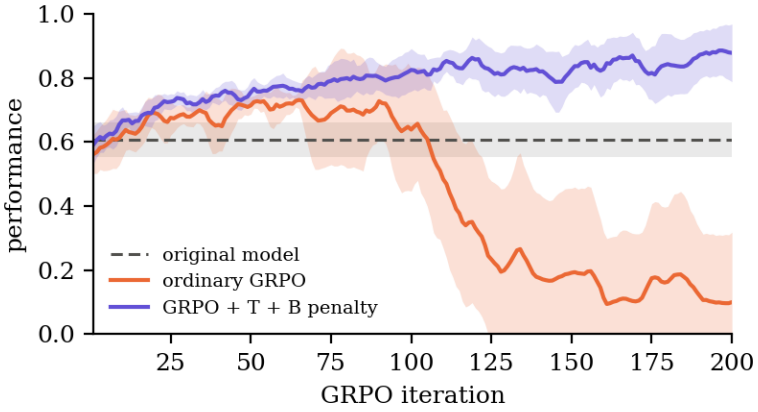


Figure 4: The original model, ordinary GRPO and GRPO with the T + B penalty, drawn as in Figure 3.

Note that ordinary GRPO lowers performance below that of the original model. This is because it discovers the reward hack, which degrades its performance on the task. It is an example of how post-training can make a model less aligned with the task it was meant to learn, a phenomenon well documented in the literature: Pan et al. (2022) find that more capable agents are more likely to exploit a misspecified reward, gaining proxy reward while losing true reward.

At the end of training, we take each seed's last 10 iterations, 320 episodes, and report the share of them that reach the goal. We say a seed **hacks the reward** if fewer than 10% of these episodes reach the goal.

In particular, compare instances of reward hacking:

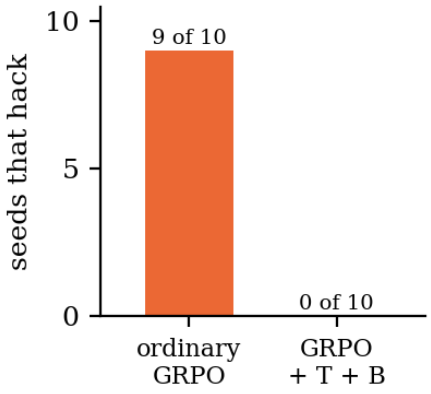


Figure 5: The number of seeds, of 10, that hack the reward.

With ordinary GRPO, 9 of 10 seeds hack the reward; with T + B penalised, none does. This difference is statistically significant at $p = 0.0001$ by a two-sided Fisher exact test. We note that over the last 10 iterations of these 9 seeds, the lower bound the monitor proves for T is between 0.74 and 0.80, so the models exhibit self-looping behaviour.

Finally, over the whole experiment:

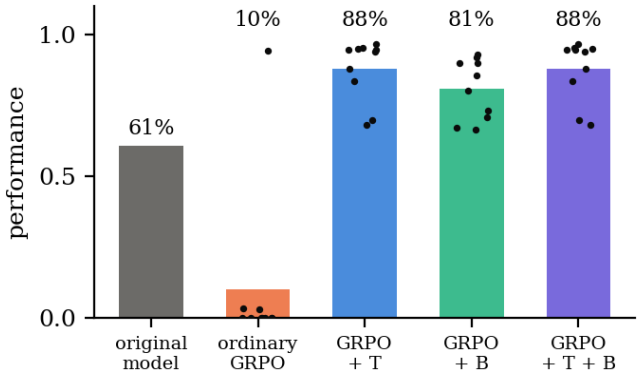


Figure 6: Performance of the original model, over 320 episodes, and at the end of each kind of training. Bars are means over 10 seeds of 320 episodes each; each dot is one seed.

Across the 10 seeds, ordinary GRPO lowers the share of episodes reaching the goal from 61% to 10%. GRPO with the T penalty raises it to 88%, with median 94%, and GRPO with the B penalty to 81%. Both differ from ordinary GRPO at $p \leq 0.002$ by a two-sided Mann–Whitney test; T and B do not differ significantly from each other at 10 seeds, $p = 0.06$.

Moreover, when we use the logic as a monitor, we note that it is able to prove the reward hack before it happens:

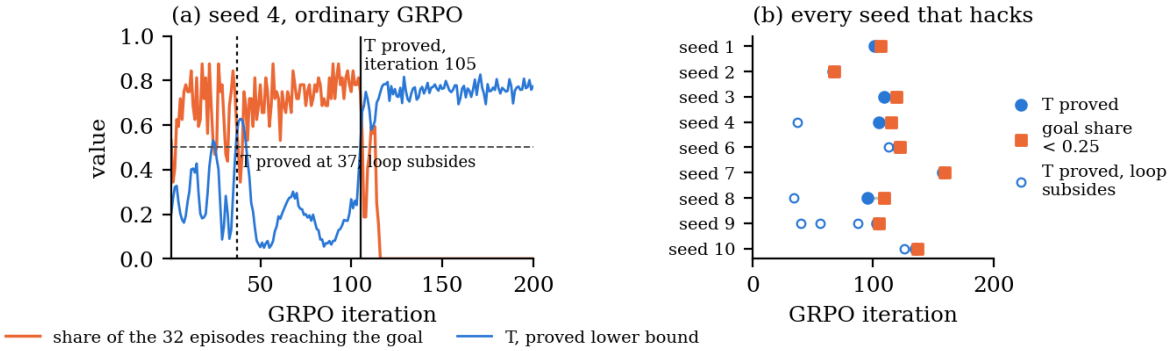


Figure 7: (a) Seed 4 with ordinary GRPO: the share of each iteration's 32 episodes that reach the goal, and the lower bound of T that the monitor proves from those episodes. The dashed line is the threshold 0.5. T is proved at iteration 37, but that loop subsides; it is proved again at iteration 105, ten iterations before the hack. (b) For each of the 9 seeds that hack: filled circle, the proof that leads into the hack, the last time before the hack that T's proved lower bound rises to 0.5 and stays there for five iterations; open circles, earlier proofs whose loops subside; square, the first iteration from which the share of episodes reaching the goal stays below 0.25 for five iterations.

Across all 9 hacking seeds, T is proved before the share of episodes reaching the goal falls below 0.25: between 1 and 14 iterations before, with median 2. In 5 seeds the monitor also proves T earlier, during loops the model then leaves, as at iteration 37 of seed 4; such a proof is an early warning rather than the hack itself. Indeed, the proof uses only the 32 episodes GRPO already samples, and therefore requires little extra computation; and by Theorem 1 its bounds are always correct: over the 40 runs, the true degree never left the proved interval in 1,847,380 checks.

4.5 Analysis

While this experiment has provided results over a simple, imaginary scenario, the same method applies wherever a language model is trained by reinforcement learning on episodes it samples. In particular, agentic tasks such as tool use are now trained with GRPO (Qian et al., 2025), and attract much attention for reward hacking: models trained on coding tasks learn to exploit flaws in the tests that grade them (Baker et al., 2025; MacDiarmid et al., 2025). The logic here provides a monitor and a penalty for such tasks: indeed, a failure such as repeating the same tool call is a loop of exactly the kind T and B detect.

Given an abstraction of the agent's situations into states, the episodes GRPO samples determine a program $N \leq M$ as in Section 4.1.1, and Theorem 1 turns them into provable bounds on what the agent does. An application to more sophisticated scenarios, including but not limited to tool use, multi-step document retrieval and agentic workflows, and research on which other formulas might capture the failures of real agents, present an interesting opportunity for more work.

5 Conclusion

In these pages, we relaxed both binary propositions and non-deterministic binary transitions to take values in $[0, 1]$, and in doing so we obtained a graded semantics for propositional dynamic logic grounded in linear algebra. A central philosophical position of the work here is: when truth comes in degrees, and we work in a probabilistic setting, that classical quantification finds an analogue in expectation. This opens up intriguing reinterpretations of ideas from classical modal and dynamic logic that give us a logical language to reason over dynamic, probabilistic systems.

Importantly, we have shown that this algebraic structure is also analytically well behaved. This suggests a bridge between formal modal specification and the computational domains, and opens the door, perhaps, to further work with potential applications to optimization or regularization against interpretable dynamic-logic formulas.

Appendix A

To motivate our choice of implication, $\varphi \to \psi = \min(1,\, 1-\varphi+\psi)$ (also known as the *Łukasiewicz implication*), picture a robot on a grid making its way to some goal according to some policy π. Suppose $x(w)$ measures the degree of danger at the current state w, and $y(w)$ measures the degree of safety. We might want to express a condition like: "if you are going to be in a dangerous place, the policy ought to lead to a safe place immediately afterward". Our syntax gives us a natural formula to express this.

Using the $\to$ connective combined with the modal diamond from Section 2, we express the condition like:

$$x \to \langle\pi\rangle y.$$

The antecedent describes how strongly a state calls for a response; the consequent describes how well the policy provides it. Importantly, evaluating the formula gives us a numerical sense of how well this property is met in expectation. Expanding the formula with $M_\pi(w, w')$ as the one-step transition probability under policy π, our semantics give

$$(\langle\pi\rangle y)(w) = \sum_{w' \in W} M_\pi(w, w') y(w'),$$

and hence

$$(x \to \langle\pi\rangle y)(w) = \min\left(1,\, 1 - x(w) + \sum_{w' \in W} M_\pi(w, w') y(w')\right).$$

Note that these semantics provide us with a graded sense of *violation*, but not a graded measure of *excess*. We choose the former to align with our intuition of requirement, but one might equally prefer the latter, and this presents an avenue for further work.

Appendix B: the policy as a program

Write $\pi(a \mid w)$ for the renormalised probability of move a at cell w, and $P(w' \mid w, a)$ for the chance that move a from w lands in w' with the agent alive. Moves are deterministic, so

$P(w' \mid w, a)$ is $1 - d(w')/10$ for the one cell w' that a leads to, where d is *danger*, and 0 elsewhere. The policy is then the program

$$M_\pi(w, w') = \sum_a \pi(a \mid w)\, P(w' \mid w, a).$$

Each row sums to the chance of surviving one step, so the leak

$$\ell(w) = 1 - \sum_{w'} M_\pi(w, w') = \sum_a \pi(a \mid w)\, \frac{d(w_a)}{10},$$

where w_a is the cell that a leads to, is exactly the chance of dying on the next step. Dying is halting, in the sense of Section 1. The goal absorbs: its row is a self-loop of weight 1, so no episode leaves it. Because the prompt depends only on the current cell, $\pi(a \mid w)$ does not depend on how the agent reached w, and M_π is a single fixed matrix for each version of the model.

In the prompt, each neighbouring cell is described by its danger and resource degrees, for instance `danger 0.5, resource 0.6`, or as `goal`, `wall` or `empty`.

References

D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, D. Mané. Concrete problems in AI safety. arXiv:1606.06565, 2016.

B. Baker, J. Huizinga, L. Gao, Z. Dou, M. Y. Guan, A. Madry, W. Zaremba, J. Pachocki, D. Farhi. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation. arXiv:2503.11926, 2025.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv:2501.12948, 2025.

L. Gao, J. Schulman, J. Hilton. Scaling laws for reward model overoptimization. *ICML*, PMLR 202:10835–10866, 2023.

E. J. Hu et al. LoRA: Low-rank adaptation of large language models. *ICLR*, 2022. arXiv:2106.09685.

N. Lambert et al. Tülu 3: Pushing frontiers in open language model post-training. arXiv:2411.15124, 2024.

A. Legay, B. Delahaye, S. Bensalem. Statistical model checking: An overview. *Runtime Verification (RV)*, pages 122–135, 2010. arXiv:1005.1327.

J. Leike, M. Martic, V. Krakovna, P. A. Ortega, T. Everitt, A. Lefrancq, L. Orseau, S. Legg. AI safety gridworlds. arXiv:1711.09883, 2017.

M. MacDiarmid, B. Wright, J. Uesato et al. Natural emergent misalignment from reward hacking in production RL. arXiv:2511.18397, 2025.

L. Ouyang et al. Training language models to follow instructions with human feedback. *NeurIPS*, 2022. arXiv:2203.02155.

A. Pan, K. Bhatia, J. Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. *ICLR*, 2022.

C. Qian, E. C. Acikgoz, Q. He, H. Wang, X. Chen, D. Hakkani-Tür, G. Tur, H. Ji. ToolRL: Reward is all tool learning needs. arXiv:2504.13958, 2025.

Qwen Team, A. Yang et al. Qwen2.5 technical report. arXiv:2412.15115, 2024.

Z. Shao et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv:2402.03300, 2024.

J. Skalse, N. H. R. Howe, D. Krasheninnikov, D. Krueger. Defining and characterizing reward gaming. *NeurIPS*, 2022. arXiv:2209.13085, as "Defining and characterizing reward hacking".